



Mecanismos de Exclusión Mutua

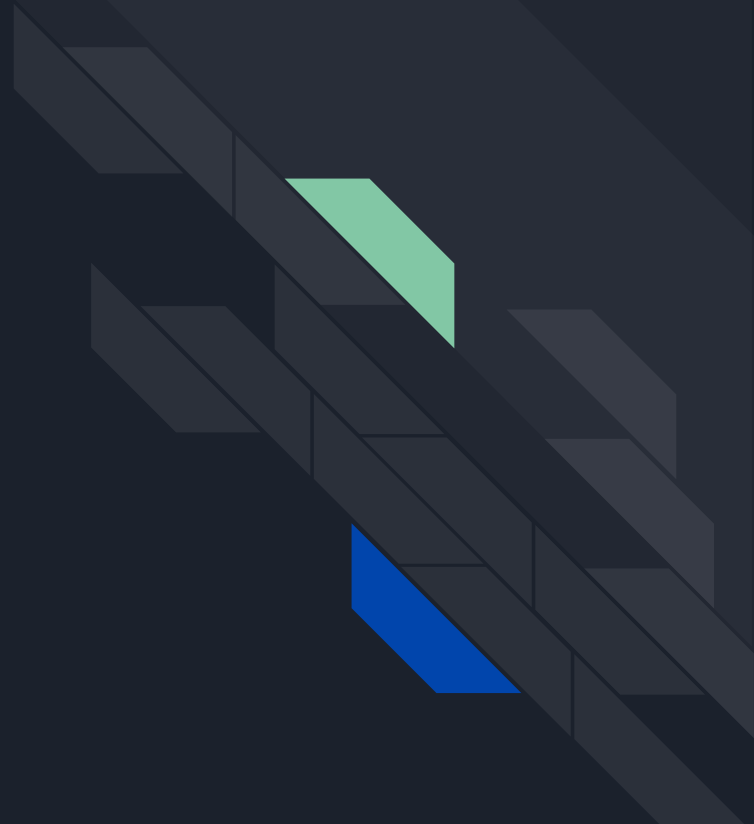
Josned Aguirre 31 083 291



¿Que es exclusion mutua?

Es la propiedad en la cual se permite que un solo hilo actúa sobre ciertos datos compartidos, para de esta manera mantener la coherencia y consistencia de datos durante la concurrencia.

Soluciones de Software para la exclusión mutua



Algoritmo de Dekker V1

```
void Hilo0(){
    /*
     | Codigo que no afecta datos compartidos
     */
    while(turnoHilo != 0){/*Espera*/};
    //Entra a la seccion critica

    //Cambio de turno
    turnoHilo = (turnoHilo+1)%totalHilos;

    //Resto del codigo
}
```

```
#include <stdio.h>
```

```
int turnoHilo;
int totalHilos;
```

```
int main(){
    turnoHilo = 0;
    totalHilos = 2;
    iniciarHilos();
}
```

```
void Hilo1(){
    /*
     | Codigo que no afecta datos compartidos
     */
    while(turnoHilo != 1){/*Espera*/};
    //Entra a la seccion critica

    //Cambio de turno
    turnoHilo = (turnoHilo+1)%totalHilos;

    //Resto del codigo
}
```

Problemas y corrida al frio

Su problema es que si ocurre un fallo durante la seccion critica, el siguiente proceso espera de manera indefinida.

Los procesos se tienen que ajustar al ritmo de ejecucion del proceso más lento

Proceso 0	Proceso 1	turno
<code>while(turno!=0){};</code>		0
	<code>while(turno!=1){};</code>	0
CS		0
<code>turno=1;</code>		1
	<code>while(turno!=1){};</code>	1
	CS	1

Algoritmo de Dekker V2

Esta version consigue arreglar el problema de que los procesos se adapten al ritmo de ejecucion del proceso más lento y tambien mantiene el estado de cada hilo de manera independiente

```
#include <stdio.h>

int estadoHilo[2] = {0,0};

int main(){
    iniciarHilos();
}
```

```
void Hilo0(){
    /*
     *Codigo que no afecta datos compartidos
     */
    while(estadoHilo[1]){/*Espera*/};
    //Entra a la seccion critica
    estadoHilo[0] = true;

    //Cambio de turno
    estadoHilo[0] = false;

    //Resto del codigo
}
```

```
void Hilo1(){
    /*
     *Codigo que no afecta datos compartidos
     */
    while(estadoHilo[0]){/*Espera*/};
    //Entra a la seccion critica
    estadoHilo[1] = true;

    //Cambio de turno
    estadoHilo[1] = false;

    //Resto del codigo
}
```

Problemas y corrida al frio

No soluciona el fallo durante la sección critica y mantiene la espera indefinida

Existe la posibilidad de que ambos procesos entren a la sección critica

Proceso 0	Proceso 1	Flag[0]	Flag[1]
<code>while(flag[1]){}</code> ;		False	False
	<code>while(flag[0]){}</code> ;	False	False
<code>flag[0]:=true;</code>		True	False
	<code>flag[1]:=true;</code>	True	True
CS		True	True
	CS	True	True

Algoritmo de Dekker V3

Cambia el orden de las líneas de código para evitar que entren a la sección crítica al mismo tiempo y así forzar la exclusión mutua

```
#include <stdio.h>

int estadoHilo[2] = {0,0};

int main(){
    iniciarHilos();
}
```

```
void Hilo0(){
    /*
     * Código que no afecta datos compartidos
     */
    estadoHilo[0] = true;
    while(estadoHilo[1]){/*Espera*/};
    //Entra a la sección crítica

    //Cambio de turno
    estadoHilo[0] = false;

    //Resto del código
}
```

```
void Hilo1(){
    /*
     * Código que no afecta datos compartidos
     */
    estadoHilo[1] = true;
    while(estadoHilo[0]){/*Espera*/};
    //Entra a la sección crítica

    //Cambio de turno
    estadoHilo[1] = false;

    //Resto del código
}
```

Problemas y corrida al frio

Sigue sin solucionar el fallo durante la sección critica y mantiene la espera indefinida

Ahora con la intencion de forzar exclusion mutua, de hecho existe la posibilidad de que ocurra que ninguno de los procesos llegue a la seccion critica

Proceso 0	Proceso 1	Flag[0]	Flag[1]
		False	False
<code>flag[0]:=true;</code>		True	False
	<code>flag[1]:=true;</code>	True	True
<code>while(flag[1]){};</code>		True	True
	<code>while(flag[0]){};</code>	True	True

Algoritmo de Dekker V4

Consigue arreglar el problema de que ninguna se realice por medio de activando y desactivando los estados durante el estado de espera con un timer aleatorio

```
#include <stdio.h>

int estadoHilo[2] = {0,0};

int main(){
    iniciarHilos();
}
```

```
void Hilo0(){
    /*
     *Codigo que no afecta datos compartidos
     */
    estadoHilo[0] = true;
    while(estadoHilo[1])
    {
        estadoHilo[0] = false;
        //Cede turno al siguiente hilo
        esperaDeTiempo();
        //Un tiempo de espera aleatorio para que pase el siguiente
        estadoHilo[0] = true;
    };
    //Entra a la seccion critica

    //Cambio de turno
    estadoHilo[0] = false;

    //Resto del codigo
}
```

```
void Hilo1(){
    /*
     *Codigo que no afecta datos compartidos
     */
    estadoHilo[1] = true;
    while(estadoHilo[0])
    {
        estadoHilo[1] = false;
        //Cede turno al siguiente hilo
        esperaDeTiempo();
        //Un tiempo de espera aleatorio para que pase el siguiente
        estadoHilo[1] = true;
    };
    //Entra a la seccion critica

    //Cambio de turno
    estadoHilo[1] = false;

    //Resto del codigo
}
```

Problemas y corrida al frio

El problema acá es que hay mucha “cortesía” por parte de los procesos y estos se pueden prolongar de manera indefinida

Proceso 0	Proceso 1	Flag[0]	Flag[1]
<code>flag[0]:=true;</code>		False	False
<code>while(flag[1])</code>	<code>flag[1]:=true;</code>	True	True
<code> flag[0]:=false;</code>	<code>while(flag[0])</code>		
<code> espera</code>	<code> flag[1]:=false;</code>	False	False
<code> flag[0]:=true;</code>	<code> espera</code>	True	True
<code>while(flag[1])</code>	<code> flag[1]:=true;</code>		
<code> flag[0]:=false;</code>	<code>while(flag[0])</code>	False	False
<code> espera</code>	<code> flag[1]:=false;</code>		
<code> flag[0]:=true;</code>	<code> espera</code>	True	True
<code>while(flag[1])</code>	<code>sigue en la espera</code>		
CS	<code> flag[1]:=true;</code>		True
<code>flag[0]:=false;</code>	<code>while(flag[0])</code>	False	

Algoritmo de Dekker V5

Reemplaza el timer aleatorio con otra variable que lleva el control de quien es el siguiente turno

```
void Hilo0(){
    /*
     | Codigo que no afecta datos compartidos
     */
    estadoHilo[0] = true;
    while(estadoHilo[1])
    {
        estadoHilo[0] = false;
        //Cede turno al siguiente hilo
        while (turnoHilo != 0){/*Espera su turno*/}
        //Un tiempo de espera aleatorio para que pase el siguiente
        estadoHilo[0] = true;
    };
    //Entra a la seccion critica

    //Cambio de turno
    turnoHilo = (turnoHilo+1)%totalHilos;
    estadoHilo[0] = false;

    //Resto del codigo
}
```

```
void Hilo1(){
    /*
     | Codigo que no afecta datos compartidos
     */
    estadoHilo[1] = true;
    while(estadoHilo[0])
    {
        estadoHilo[1] = false;
        //Cede turno al siguiente hilo
        while (turnoHilo != 1){/*Espera su turno*/}
        //Un tiempo de espera aleatorio para que pase el siguiente
        estadoHilo[1] = true;
    };
    //Entra a la seccion critica

    //Cambio de turno
    turnoHilo = (turnoHilo+1)%totalHilos;
    estadoHilo[1] = false;

    //Resto del codigo
}
```

```
#include <stdio.h>
```

```
int estadoHilo[2] = {0,0};
int turnoHilo;
int totalHilos;
```

```
int main(){
    turnoHilo = 0;
    totalHilos = 2;
    iniciarHilos();
}
```

Problemas y corrida al frio

Soluciona todos los problemas anteriores, pero el problema actual es que no es tan simple el algoritmo, ni elegante, tampoco es tan facil de llevar el control y escalarlo tampoco es tan sencillo comparado a otros

Proceso 0	Proceso 1	Flag[0]	Flag[1]	Turno
		False	False	0
flag[0]=true;		True		
while(flag[1])	flag[1]=true;		True	
flag[0]=false;	while(flag[0])	False		
while(turno!=0);	flag[1]=false;		False	
flag[0]=true;	while(turno!=1);	True		
while(flag[1])	while(turno!=1);			
CS	while(turno!=1);			
turno=1;	while(turno!=1);			1
flag[0]=false;	flag[1]=true;	False	True	
	while(flag[0])			

Algoritmo de Peterson

Es una variación del algoritmo anterior el cual lo convierte en una solución más simple, elegante y más sencilla de escalar

```
void Hilo0(){
    /*
     | Codigo que no afecta datos compartidos
     */
    estadoHilo[0] = true;
    turnoHilo = 1;
    while(estadoHilo[1] && turnoHilo == 1){/*Espera*/};
    //Entra a la seccion critica

    //Cambio de turno
    estadoHilo[0] = false;

    //Resto del codigo
}
```

```
#include <stdio.h>
```

```
int estadoHilo[2] = {0,0};
int turnoHilo;
int totalHilos;
```

```
int main(){
    turnoHilo = 0;
    totalHilos = 2;
    iniciarHilos();
}
```

Problemas y corrida al frío

La eficiencia de la ejecución puede verse notablemente degradada debido a que se limita la capacidad del procesador para intercalar programas.

No funciona adecuadamente para multiprocesadores

Proceso 0	Proceso 1	Flag[0]	Flag[1]	Turno
		False	False	0
Flag[0]=True;		True	False	0
	Flag[1]=True;	True	True	0
Turno=1		True	True	1
	Turno=0	True	True	0
while (flag[1]&&turno==1){};		True	True	0
	while (flag[0]&&turno==0){};	True	True	0
CS		True	True	0
Flag[0]=False		False	True	0
	while (flag[0]&&turno==0){};	False	True	0
	CS	False	True	0

Algoritmo de Panaderia de Lamport

Este es un algoritmo realizado con el fin de dar la misma ventaja en base a la escalabilidad atendiendo los procesos como en una panadería

```
int numero[N];  
int seleccionado[N];
```

```
int main(){  
    //Inicializa los valores de seleccionado en false  
    // y numero en 0  
    iniciarHilos();  
}
```

```
void HiloI(){  
    /*  
     | Codigo que no afecta datos compartidos  
     */  
    seleccionado[i] = true;  
    numero[i] = max(numero) + 1;  
    seleccionado[i] = false;  
    for (int j = 0; j < N; j++)  
    {  
        while (seleccionado[j]){/*Espera a que termine de seleccionar*/};  
        while (numero[j] != 0 &&  
            || (numero[j] < numero[i] || (numero[j] == numero[i] && j < i) )){/* Espera el turno del que viene */}  
        }  
        //Entra a la seccion critica  
  
        //Cambio de turno  
        numero[i] = 0;  
  
        //Resto del codigo  
    }  
}
```

Problemas y corrida al frío

Al igual que el anterior no es el mejor para multiprocesadores, es algo complejo de implementar y si bien es mas escalable, puede degradar la eficiencia por intercalar entre programas

Proc[0]	Proc[1]	Proc[2]	num[0]	num[1]	num[2]	selec[0]	selec[1]	selec[2]
			0	0	0	false	false	false
s[0]=true						true		
	s[1]=true						true	
		s[2]=true						true
max(n[0],n[1])								
	max(n[0],n[1])							
		max(n[0],n[1])						
n[0] =max(n[1],n[2])+1			1					
n[0] = 1	n[1]=max(n[1],n[2])+1			1				
	n[1] = 1	n[2] =max(n[1],n[2])+1			2			
s[0]=false		n[2] = 2				false		

Corrida al frío

Proc[0]	Proc[1]	Proc[2]	num[0]	num[1]	num[2]	selec[0]	selec[1]	selec[2]
	s[1]=false		1	1	2	false	false	true
		s[2]=false						false
mientras(s[1])								
	mientras(s[0])							
		mientras(s[0])						
mientras(n[1] != 0 && (n[1] < n[0] (n[1] == n[0] && 1 < 0)))								
	mientras(n[0] != 0 && (n[0] < n[1] (n[0] == n[1] && 0 < 1)))							
	No puede pasar	mientras(n[0] != 0 && (n[0] < n[2] (n[0] == n[2] && 0 < 2)))						
mientras(s[2])		No puede pasar						

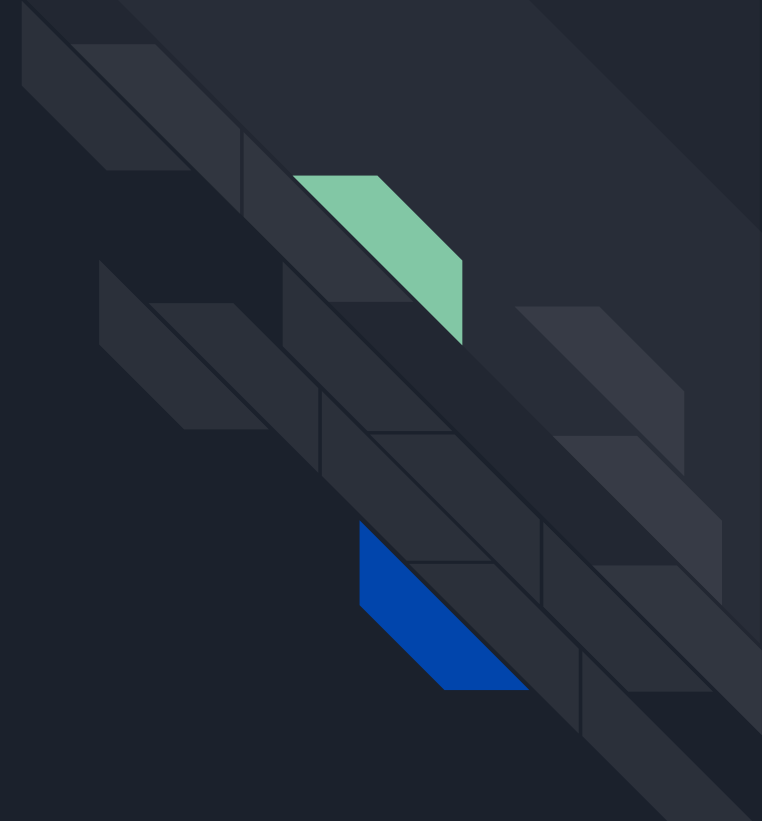
Corrida al frío

Proc[0]	Proc[1]	Proc[2]	num[0]	num[1]	num[2]	selec[0]	selec[1]	selec[2]
<code>mientras(n[2] != 0 && (n[2] < n[0] (n[2] == n[0] && 2 < 0)))</code>			1	1	2	false	false	false
CS								
<code>n[0] = 0</code>			0					
	<code>mientras(n[0] != 0 && (n[0] < n[1] (n[0] == n[1] && 0 < 1)))</code>							
		<code>mientras(n[0] != 0 && (n[0] < n[2] (n[0] == n[2] && 0 < 2)))</code>						
	<code>mientras(n[2] != 0 && (n[2] < n[1] (n[2] == n[1] && 2 < 1)))</code>							
		<code>mientras(n[1] != 0 && (n[1] < n[2] (n[1] == n[2] && 1 < 2)))</code>						
	CS	no puede pasar						

Corrida al frío

Proc[0]	Proc[1]	Proc[2]	num[0]	num[1]	num[2]	selec[0]	selec[1]	selec[2]
	$n[1] = 0$		0	0	2	false	false	false
		<pre>mientras($n[1] \neq 0$ && ($n[1] < n[2]$ ($n[1] == n[2]$ && $1 < 2$)))</pre>						
		CS						
		$n[1] = 0$			0			

SOLUCIONES DE HARDWARE PARA EXCLUSIÓN MUTUA





Estas son instrucciones que sirven para la exclusion mutua dentro del propio hardware por medio de tener los accesos a las posiciones de memoria

Se diseñaron 2 instrucciones maquinas a nivel atómico (es indivisible y no se pueden observar los estados intermedios de estos), estas 2 se ejecutan como una sola accion

La ejecucion de estas instrucciones como se mencionó es mutuamente exclusiva incluso tomando en cuenta varios CPUs

Pero tambien necesitan complementarse con otros mecanismos para resolver el problema de la seccion critica

Test and Set

La instrucción examina el valor de su argumento, i. Si el valor es 0, lo cambia por 1 y devuelve cierto. En otro caso, el valor no se modifica y se devuelve falso. La función Comparar y Fijar se ejecuta inmediatamente en su totalidad, es decir, no está sujeta a interrupciones.

```
void HiloI(){
    /*
     | Codigo que no afecta datos compartidos
     */
    while (testAndSet(&control));

    //Entra a la seccion critica

    //Cambio de turno
    control = 0;
    //Resto del codigo
}
```

```
int control = 0;
```

```
int main(){
    iniciarHilos();
}
```

```
int testAndSet(int *x){
    if(*x == 0){
        *x = 1;
        return 1;
    }
    return 0;
}
```

Corrida al frío

P0	P1	P2	b	r ₀	r ₁	r ₂
			0		0	0
do{ testandset b,r } while(r==0);			1	1		
	do{ testandset b,r } while(r==0);		1		0	
		do{ testandset b,r } while(r==0);	1			0
SC						
b=0;			0			
	do{ testandset b,r } while(r==0);		1		1	
		do{ testandset b,r } while(r==0);	1			0
	SC		1			



Ventajas y Desventajas

Ventajas

Mantenemos la exclusión mutua

Es una instrucción atómica por lo que no tiene interrupciones

Es simple y fácil de implementar

Puede ser utilizado para dar soporte a múltiples secciones críticas

Desventajas

Se emplea espera activa

Es posible la inanición.

Es posible el interbloqueo.

Compare and Swap

Es otra instruccion atomica, esta siempre devuelve el valor original por medio de otra variable, pero cuando ve que el valor original es igual al esperado, se actualiza la variable del valor original para obtener un nuevo valor.

```
void HiloI(){  
    /*  
    | Codigo que no afecta datos compartidos  
    */  
    while (compareAndSwap(&control,0,1));  
  
    //Entra a la seccion critica  
  
    //Cambio de turno  
    control = 0;  
    //Resto del codigo  
}
```

```
int control = 0;  
  
int main(){  
    | iniciarHilos();  
}
```

```
int compareAndSwap(int *valor, int esperado, int nuevo){  
    int tmp = *valor;  
    if(*valor == esperado)  
        | *valor = nuevo;  
    return tmp;  
}
```

Corrida al frío



Proc[0]	Proc[1]	control
		0
CaS(control,0,1) = 0;		1
	CaS(control,0,1) = 1;	1
CS		1
control = 0		0
	CaS(control,0,1) = 0;	1
	CS	1
	control = 0	0



Ventajas y Desventajas

Ventajas

Mantenemos la exclusión mutua

Es una instrucción atómica por lo que no tiene interrupciones

Previene choque entre multiprocesadores

Uso más optimo del CPU

Desventajas

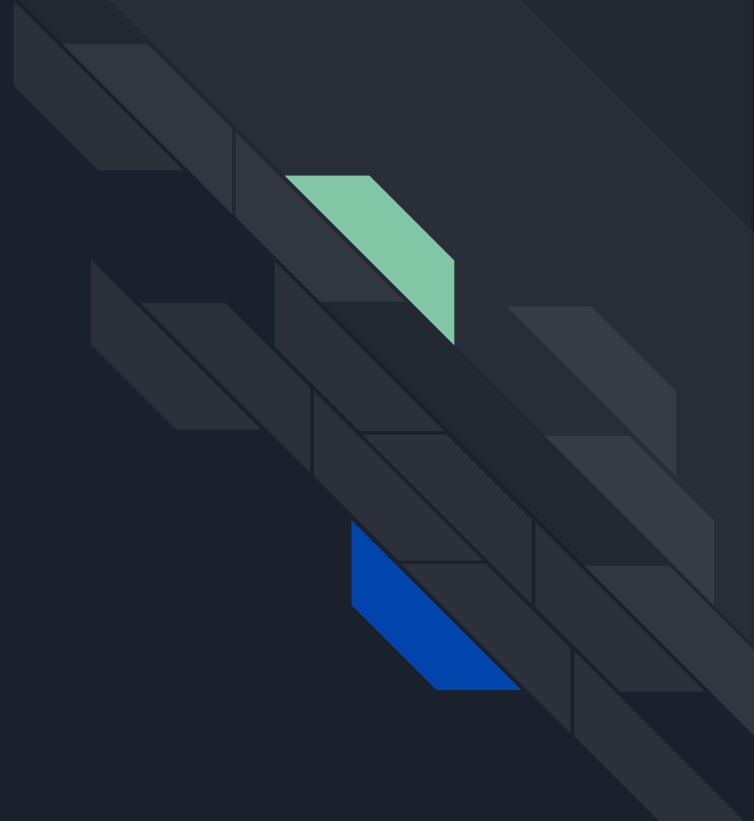
Se emplea espera activa

Es posible la inanición.

Es posible el interbloqueo.

Es más complejo

Algoritmos clásicos



Semáforo Productor-Consumidor

El planteamiento general es el siguiente: Uno o más productores generan cierto tipos de datos (registros, caracteres) y los sitúan en un buffer. Un único consumidor saca elementos del buffer de uno en uno. El sistema está obligado a impedir la superposición de operaciones sobre el buffer. Se usará un semáforo de enteros

```
typedef struct  
{  
    int contador;  
    queue colaBloqueados;  
}SEMAPHORE;
```

```
void Wait(SEMAPHORE *s){  
    s->contador--;  
    if(s->contador < 0){  
        /*poner proceso en s.colaBloqueados  
        bloquear el proceso*/  
    }  
}
```

```
void Signal(SEMAPHORE *s){  
    s->contador++;  
    if(s->contador < 0){  
        /*quitar el proceso de s.colaBloqueados  
        poner en la cola de listos*/  
    }  
}
```

Semáforo Productor-Consumidor Buffer ilimitado

```
int main(){
    initSemE(s,1);
    initSemE(n,0);
    IniciarHilos(Produtor(),Consumidor());
}

void Produtor(){
    while(true){
        producir();
        /*
         *Codigo que no afecta datos compartidos
         */
        Wait(s);
        //Entra a la seccion critica
        anadir();
        //Cambio de turno
        Signal(s);
        Signal(n);
        //Resto del codigo
    }
}

void Consumidor(){
    while(true){
        /*
         *Codigo que no afecta datos compartidos
         */
        Wait(n);
        Wait(s);
        //Entra a la seccion critica
        tomar();
        //Cambio de turno
        Signal(s);
        //Resto del codigo
        consumir();
    }
}
```

Corrida al frío



Productor	Consumidor	n.count	s.count
		0	1
wait(s)			0
	wait(n)	-1	
CS			
Signal(s)			1
Signal(n)		0	
Fin	Wait(n)		
	Wait(s)		0
	CS		
	Signal(s)		1

Semáforo Productor-Consumidor Buffer limitado

```
SEMAPHORE n;  
SEMAPHORE s;  
SEMAPHORE buffer;
```

```
int main(){  
    initSemE(s,1);  
    initSemE(n,0);  
    initSemE(buffer,BUFFER_SIZE);  
    IniciarHilos(Productor(),Consumidor());  
}
```

```
void Productor(){  
    while(true){  
        producir();  
        /*  
         *Codigo que no afecta datos compartidos  
         */  
        Wait(buffer);  
        Wait(s);  
        //Entra a la seccion critica  
        anadir();  
        //Cambio de turno  
        Signal(s);  
        Signal(n);  
        //Resto del codigo  
    }  
}
```

```
void Consumidor(){  
    while(true){  
        /*  
         *Codigo que no afecta datos compartidos  
         */  
        Wait(n);  
        Wait(s);  
        //Entra a la seccion critica  
        tomar();  
        //Cambio de turno  
        Signal(s);  
        Signal(buffer);  
        //Resto del codigo  
        consumir();  
    }  
}
```

Corrida al frío



Productor	Consumidor	n.count	s.count	buffer
		0	1	3
wait(Buffer)				2
Wait(s)			0	
CS				
Signal(s)			1	
Signal(n)		1		
wait(Buffer)				1
	Wait(n)	0		
wait(s)			0	
	wait(s)		-1	
CS				

Corrida al frío



Productor	Consumidor	n.count	s.count	buffer
		0	-1	1
Signal(s)			0	
	wait(s)			
	CS			
Signal(n)		1		
	Signal(s)		1	
	Signal(buffer)			

Barbero dormilón

```
SEMAPHORE maxCapacidad, sofa, sillaBarbero, coord,  
clienteListo, terminado, recibo, dejarSilla, pago,  
recibo;
```

```
int main(){  
    initSemE(maxCapacidad,20);  
    initSemE(sofa,4);  
    initSemE(sillaBarbero,3);  
    initSemE(coord,3);  
    initSemE(clienteListo,0);  
    initSemE(terminado,0);  
    initSemE(dejarSilla,0);  
    initSemE(pago,0);  
    initSemE(recibo,0);  
    IniciarHilos[ cliente()/*N veces */,  
                 barbero()/*3 veces */,  
                 cajero()];  
}
```

```
void Cliente(){  
    while(true){  
        Wait(maxCapacidad);  
        //entrar();  
        Wait(sofa);  
        //sentarse();  
        Wait(sillaBarbero);  
        //salirSofa();  
        Signal(sofa);  
        //sentarSilla();  
        Signal(clienteListo);  
        Wait(terminado);  
        //salirSilla();  
        Signal(dejarSilla);  
        //pagar();  
        Signal(pago);  
        Wait(recibo);  
        //salirTienda();  
        Signal(maxCapacidad);  
    }  
}
```

```
void Barbero(){  
    while(true){  
        Wait(clienteListo);  
        Wait(coord);  
        //cortarPelo();  
        Signal(coord);  
        Signal(terminado);  
        Wait(dejarSilla);  
        Signal(sillaBarbero);  
    }  
}  
  
void Cajero(){  
    while(true){  
        Wait(pago);  
        Wait(coord);  
        //aceptarPago();  
        Signal(coord);  
        Signal(recibo);  
    }  
}
```

Barbero dormilón

```
SEMAPHORE maxCapacidad, sofa, sillaBarbero, coord,  
clienteListo, terminado, recibo, dejarSilla, pago,  
recibo;
```

```
int main(){  
    initSemE(maxCapacidad,20);  
    initSemE(sofa,4);  
    initSemE(sillaBarbero,3);  
    initSemE(coord,3);  
    initSemE(clienteListo,0);  
    initSemE(terminado,0);  
    initSemE(dejarSilla,0);  
    initSemE(pago,0);  
    initSemE(recibo,0);  
    IniciarHilos[ cliente()/*N veces */,  
                  barbero()/*3 veces */,  
                  cajero()];  
}
```

```
void Cliente(){  
    while(true){  
        Wait(maxCapacidad);  
        //entrar();  
        Wait(sofa);  
        //sentarse();  
        Wait(sillaBarbero);  
        //salirSofa();  
        Signal(sofa);  
        //sentarSilla();  
        Signal(clienteListo);  
        Wait(terminado);  
        //salirSilla();  
        Signal(dejarSilla);  
        //pagar();  
        Signal(pago);  
        Wait(recibo);  
        //salirTienda();  
        Signal(maxCapacidad);  
    }  
}
```

```
void Barbero(){  
    while(true){  
        Wait(clienteListo);  
        Wait(coord);  
        //cortarPelo();  
        Signal(coord);  
        Signal(terminado);  
        Wait(dejarSilla);  
        Signal(sillaBarbero);  
    }  
}  
  
void Cajero(){  
    while(true){  
        Wait(pago);  
        Wait(coord);  
        //aceptarPago();  
        Signal(coord);  
        Signal(recibo);  
    }  
}
```

Lectores-Escritores

```
void Lector(){
    while(true){
        Wait(z);
        Wait(lecturaPermitida);
        Wait(x);
        ++contLector;
        if(contLector == 1)
            Wait(escrituraPermitida);
        Signal(x);
        Signal(lecturaPermitida);
        Signal(z);
        //Lectura();
        Wait(x);
        --contLector;
        if(contLector == 0)
            Signal(escrituraPermitida);
        Signal(x);
    }
}
```

```
void Escritor(){
    while(true){
        Wait(y);
        ++contEscritores;
        if(contEscritores == 1)
            Wait(lecturaPermitida);
        Signal(y);
        Wait(escrituraPermitida);
        //Escribir();
        Signal(escrituraPermitida);
        Wait(y);
        --contEscritores;
        if(contEscritores == 0)
            Signal(lecturaPermitida);
        Signal(y);
    }
}
```

```
int contLector, contEscritores;
SEMAPHORE x, y, z, escrituraPermitida, lecturaPermitida;

int main(){
    contLector = contEscritores = 0;
    initSemE(x, 1);
    initSemE(y, 1);
    initSemE(z, 1);
    initSemE(escrituraPermitida, 1);
    initSemE(escrituraPermitida, 1);
    IniciarHilos(Lector()/*N veces */,
                Escritor()/*M veces */);
}
```

Filósofos comensales

```
void Filosofo(int i){
    while(true){
        piensa();
        /*
         *Codigo que no afecta datos compartidos
         */
        Wait(comedor);
        Wait(tenedor[i]);
        wait(tenedor[(i+1)%5]);
        //Entra a la seccion critica
        come();
        //Cambio de turno
        Signal(tenedor[(i+1)%5]);
        Signal(tenedor[i]);
        Signal(comedor);
        //Resto del codigo
    }
}
```

```
SEMAPHORE tenedor[5];
SEMAPHORE comedor;
int i;

int main(){
    initSemE(comedor,4);
    for(int x = 0 ; x < 5;++x){
        initSemE(tenedor[x],1);
    }
    IniciarHilos(Filosofo(0),Filosofo(1),Filosofo(2),
    Filosofo(3),Filosofo(4));
}
```



Monitores

Los monitores son estructuras de un lenguaje de programación que ofrece una equivalente a los semáforos y que son más fáciles de controlar. Esto permite poner cierres de monitor a objetos cualquiera. En particular, para algo parecido a una lista enlazada, se puede querer bloquear todas las listas enlazadas con un solo cierre o bien tener un cierre para cada elemento de cada lista

Monitor Productor-Consumidor Buffer limitado

```
class monitor {
    char buffer[N]; /* espacio para N datos */
    int dentro, fuera; /* punteros al buffer */
    int cuenta; /* número de datos en el buffer */
    cond nolleno, novacio; /* variables condición para sincronizar */
    void anyadir (char x) {
        if (cuenta == N)
            cwait(nolleno); /* buffer lleno, evitar desbordar */
        buffer[dentro] = x;
        dentro = (dentro + 1) % N;
        ++cuenta; /* un dato más en el buffer */
        csignal(novacio); /* retoma algún consumidor en espera */
    }

    void extraer (char x){
        if (cuenta == 0)
            cwait(novacio); /* buffer vacío, evitar consumo */
        x = buffer[fuera];
        fuera = (fuera + 1) % N;
        --cuenta; /* un dato menos en el buffer */
        csignal(nolleno); /* retoma algún productor en espera */
    }

    void construir(){ /* cuerpo del monitor */
        dentro = 0; fuera = 0; cuenta = 0; /* buffer inicialmente vacío */
    }
}
```

```
void productor()
{
    char x;
    while (true)
    {
        producir(x);
        monitor.anyadir(x);
    }
}

void consumidor()
{
    char x;
    while (true)
    {
        monitor.extraer(x);
        consumir(x);
    }
}

void main()
{
    IniciarHilos(productor, consumidor);
}
```

Lectores-Escritores

```
class monitor{
    int lectores = 0, escritores = 0; // Número de lectores activos
    cond puedeLeer, puedeEscribir;

    void empiezaLeer() {
        if (escritores > 0)
            cwait(puedeLeer);
        ++lectores;
        csignal(puedeLeer);
    }

    void terminaLeer() {
        --lectores;
        if (lectores == 0) {
            csignal(puedeEscribir);
        }
    }

    void empiezaEscribir() {
        if (lectores > 0 || escritores > 0)
            cwait(puedeEscribir);
        ++escritores;
    }

    void terminaEscribir() {
        if(escritores < 1)
            csignal(puedeLeer);
        --escritores;
        csignal(puedeEscribir);
    }
}
```

```
int main(){
    IniciarHilos(Lector()/*N veces */,
                Escritor()/*M veces*/);
}

void Lector(){
    while (true) {
        monitor.empiezaLeer();
        // Leer datos
        monitor.terminaLeer();
    }
}

void Escritor(){
    while (true) {
        monitor.empiezaEscribir();
        // Escribir datos
        monitor.terminaEscribir();
    }
}
```

Barbero dormilón

```
int main(){
    IniciarHilos(cliente()/*N veces */,
                barbero()/*3 veces*/);
}

class monitor {
    int clientesEsperando = 0;
    int barberosDisponibles[3] = {true,true,true}; // Número de barberos disponibles
    cond barberoListo[3], nolleno;

    void entraCliente(int barbero) {
        if (clientesEsperando > NUM_SILLAS) { cwait(nolleno);
            ++clientesEsperando;
            csignal(barberoListo[barbero]);
        }
    }

    void atiendeCliente(int barbero) {
        while (true) {
            if(!barberosDisponibles[barbero])
                cwait(barberoListo[barbero]);
            barberosDisponibles[barbero] = false;
            --clientesEsperando;
            csignal(nolleno);
        }
    }
}
```

```
void Barbero(){
    while (true) {
        monitor.atiendeCliente();
        corte();
    }
}

void Cliente(){
    monitor.entraCliente();
}
```

Filósofos comensales

```
class monitor {
    cond TenedorListo[5]; /* variable de condición para sincronizar */
    bool tenedor [5] = {true,true,true,true,true}; /* disponibilidad de cada tenedor */
    void obtiene_tenedores (int id_pr){
        int izquierdo = id_pr;
        int derecho = (++id_pr) % 5;
        /* concede el tenedor izquierdo */
        if (!tenedor[izquierdo])
            cwait(TenedorListo[izquierdo]); /* encola en variable de cond
        tenedor[izquierdo] = false;
        /* concede el tenedor derecho */
        if (!tenedor[derecho])
            cwait(TenedorListo[derecho]); /* encola en variable de condic
        tenedor[derecho] = false;
    }
    void libera_tenedores (int id_pr){
        int izquierdo = id_pr;
        int derecho = (id_pr++) % 5;
        /* libera el tenedor izquierdo */
        if (TenedorListo[izquierdo] == NULL) /* nadie espera por este t
            tenedor[izquierdo] = true;
        else /* despierta a un proceso que espera por este tenedor */
            csignal(TenedorListo[izquierdo]);
        /*libera el tenedor derecho*/
        if (TenedorListo[derecho] == NULL) /* nadie espera por este ten
            tenedor[derecho] = true;
        else /* despierta a un proceso que espera por este tenedor */
            csignal(TenedorListo[derecho]);
    }
}
```

```
void filosofo(int k) /* los cinco clientes filósofos */
{
    while (true)
    {
        //piensa;
        monitor.obtiene_tenedores(k); /* cliente solicita do
        //<come espaguetis>;
        monitor.libera_tenedores(k); /* cliente libera tened
    }
}

void main()
{
    IniciarHilos(filosofo(0),...,filosofo(4));
}
```



Paso de mensajes

Un método posible para ofrecer ambas funciones es el paso de mensajes. El paso de mensajes tiene la ventaja adicional de que se presta a ser implementado en sistemas distribuidos, así como en sistemas multiprocesador y monoprocesador de memoria compartida. El envío y recepción de mensajes es una forma estándar de comunicación

entre nodos de una red de computadores

Un mensaje es una colección de información que puede ser intercambiada

entre un proceso emisor y receptor, puede contener datos, ordenes de ejecución e incluso código a transmitir entre dos o mas procesos

Monitor Productor-Consumidor Buffer limitado

```
#define capacidad 100
buzon puedeProducir, puedeConsumir;

void main()
{
    crearBuzon(puedeProducir);
    crearBuzon(puedeConsumir);
    for (int i = 0; i < capacidad; i++)
        send(puedeProducir, NULL);

    IniciarHilos(Productor(), Consumidor());
}
```

```
void Productor(){
    mensaje msjp;
    while(true){
        recieveB(puedeProducir, msjp);
        msjp = producir();
        send(puedeConsumir, msjp);
    }
}

void Consumidor(){
    mensaje msjc;
    while(true){
        recieveB(puedeConsumir, msjc);
        consumir(msjp);
        send(puedeProducir, msjc);
    }
}
```

Barbero dormilón

buzon solicitudDeCorte,sofaLibre,terminado,pago,finCorte,solicit

```
int main(){
    IniciarHilos(ClienteI()/*N veces
                BarberoJ()/*M veces
                Controlador());
}

void ClienteI(){
    mensaje msjC;
    int i;
    while (true) {
        msjC = i;
        recieveB(sofaLibre,msjC);
        sendB(solicitudDeCorte,msjC);
        recieveB(finCorte,msjC);
        send(pago,msjC);
        msjC = i;
        send(terminado,msjC);
    }
}
```

```
void BarberoJ(){
    mensaje msjB;
    int j;
    while (true) {
        msjB = j;
        recieveB(solicitudDeCorte,msjB);
        corte();
        send(finCorte,msjB);
        msjB = j;
        send(terminado,msjB);
    }
}
```

```
void Controlador(){
    mensaje msj;
    int barberosDisponibles = 3, maxSofa = 4, contador = 0;
    while (true){
        if(contador <= maxSofa){
            if(!vacio(solicitudDeCorteJ)){
                send(sofaLibre,msj);
                ClienteJ.id = msj.id;
                ++contador;
            }
            else{
                send(solicitudDeCorteJ,msj);
                --barberosDisponibles;
            }
        }
        else if(!vacio(pago)){
            recieve(pago,msj);
            Pago(msj);
        }
        else if(!vacio(terminado)){
            recieve(terminado,msj);
            --contador;
            ++barberosDisponibles;
        }
    }
}
```

Lectores-Escritores

```
void IniciarHilos();
```

```
buzon buzonLi, buzonEj, solicitudLectura, solicitudEscritura, terminado;
```

```
int main(){
```

```
    IniciarHilos(LectorI()/*N veces */,  
                EscritorJ()/*M veces */,  
                Controlador());
```

```
void LectorI(){
```

```
    mensaje msjL;  
    int i;  
    while (true) {  
        msjL = i;  
        send(solicitudLectura, msjL);  
        recieveB(buzonLi, msjL);  
        leer();  
        msjL = i;  
        send(terminado, msjL);  
    }
```

```
void EscritorJ(){
```

```
    mensaje msjE;  
    int j;  
    while (true) {  
        msjE = j;  
        send(solicitudEscritura, msjE);  
        recieveB(buzonEj, msjE);  
        escribir();  
        msjE = j;  
        send(terminado, msjE);  
    }
```

```
void Controlador(){
```

```
    mensaje msj;  
    int contador = 100;  
    while (true){  
        if(cont > 0){  
            if(!vacio(terminado)){  
                recieveB(terminado, msj);  
                ++contador;  
            }  
            else if(!vacio(solicitudEscritura)){  
                recieveB(solicitudEscritura, msj);  
                EscritorJ.id = msj.id;  
                contador = contador - 100;  
            }  
            else if(!vacio(solicitudLectura)){  
                recieveB(solicitudLectura, msj);  
                --contador;  
                send(msj.id, "OK");  
            }  
        }  
        if(contador == 0){  
            send(escribir.id, "OK");  
            recieveB(terminado, msj);  
            contador = 100; }  
        while (contador < 0) {  
            recieveB(terminado, msj);  
            ++contador;  
        }  
    }
```



Bibliografía

SISTEMAS OPERATIVOS Segunda edición William Stallings

OPERATING SYSTEM CONCEPTS Silberzchatz

Sistemas operativos Aspectos internos y principios de diseño Quinta Edición WILLIAM STALLINGS

[Compare-and-swap - Wikipedia](#)

<https://www.infor.uva.es/~cllamas/concurr/pract98/sisos13/index.html>

<https://takeuforward.org/operating-system/producer-consumer-problem/>

[SO Problemas de concurrencia - YouTube](#)

[Operating Systems](#)

[\(PDF\) System Modelling With Petri Nets](#)

https://ics.uci.edu/~ardalan/courses/os/lectures/os_lecture_4.pdf

GRACIAS

Me faje aun así la lámina parezca de poco presupuesto.

